

```
# Each dictionary in specs has keys 'sizeChange', 'xChange',  
# 'yChange', and 'angleChange'. The size, x, and y changes  
# are multiplied by the size parameter. The x change and y  
# change are added to the turtle's current position. The angle  
# change is added to the turtle's current heading.  
sizeCh = spec.get('sizeChange', 1.0)  
xCh = spec.get('xChange', 0.0)  
yCh = spec.get('yChange', 0.0)  
angleCh = spec.get('angleChange', 0.0)
```

The `for` loop assigns an individual specification dictionary in the `specs` list to the loop variable `spec` on each iteration of the loop. The `get()` dictionary method calls pull the values for the `sizeChange`, `xChange`, `yChange`, and `angleChange` keys from this dictionary and assign them to the shorter-named `sizeCh`, `xCh`, `yCh`, and `angleCh` variables. The `get()` method substitutes a default value if the key doesn't exist in the dictionary.

Next, the turtle's position and heading are reset to the values indicated when `drawFractal()` was first called. This ensures that the recursive calls from previous loop iterations don't leave the turtle in some other place. Then the heading and position are changed according to the `angleCh`, `xCh`, and `yCh` variables:

Python

```
# Reset the turtle to the shape's starting point:  
turtle.goto(initialX, initialY)  
turtle.setheading(initialHeading + angleCh)  
  
turtle.forward(size * xCh)  
turtle.left(90)  
turtle.forward(size * yCh)  
turtle.right(90)
```

The x-change and y-change positions are expressed relative to the turtle's current heading. If the turtle's heading is 0, the turtle's relative x-axis is the same as the actual x-axis on the screen. However, if the turtle's heading is, say, 45, the turtle's relative x-axis is at a 45-degree tilt. Moving "right" along the turtle's relative x-axis would then move at an up-right angle.

This is why moving forward by `size * xCh` moves the turtle along its relative x-axis. If `xCh` is negative, `turtle.forward()` moves left along the turtle's